

Física Computacional

ano letivo: 2021/2022

docente: Nuno Castro [nuno.castro@fisica.uminho.pt]

Licenciatura em Física e Licenciatura / Mestrado Integrado em Engenharia Física

2ª aula (14/10/2021)



“Finite” Number Representations

- ◆ Multiple meanings for “finite”
- ◆ Bits = *binary integer* = $(0, 1) = (\uparrow, \downarrow)$
- ◆ All computer numbers ultimately binary
- ◆ Integers: N bits $\Rightarrow 2^N$ max
- ◆ $\Rightarrow$ Limited range $[0, 2^{N-1}]$ (1 sign, 32 b $\Rightarrow 10^9$)
- ◆ Binary: 110010010101: OK if computer
- ◆ People: *octal, decimal, hexadecimal* ()
- ◆ Decimal: nice but reduces precision, final output

Finite Number Representations (2)

- ◆ **"Word length" = # bits for stored number**
- ◆ **1 byte = 1 B = 8 bits** ()
- ◆ **Careful: 1 K = 1 KB = 2^{10} B = 1024 B**
- ◆ **1 byte $\approx$ memory for 1 character ("a")**
- ◆ **1 typed page $\approx$ 3 KB**
- ◆ **1st PCs: 8-b words ($2^7 = 128$)**
- ◆ **"Overflow": too large a number; "Underflow"**
- ◆ **Now: 32, 64 bit PC, fastest processing**
- ◆ **$2^{31} \approx 2 \times 10^9$: OK for banks, signals, not science**

IEEE Number Types

<i>Name</i>	<i>Type</i>	<i>Bits</i>	<i>Range</i>
boolean	logical	1	<i>true or false</i>
char	string	16	'\u0000' ↔ '\uFFFF'
byte	integer	8	-128 ↔ +127
short	integer	16	-32,768 ↔ +32,767
int	integer	32	-2,147,483,648 ↔ +2,147,483,647
long	integer	64	-9,223,372,036,854,775,808 ↔ +9,223,372,036,854,775,807
float	floating point	32	$1.401298 \times 10^{-45} \leftrightarrow 3.402923 \times 10^{+38}$
double	floating point	64	$4.94065645841246544 \times 10^{-324} \leftrightarrow 1.7976931348623157 \times 10^{+308}$

IEEE Floating Point

- ◆ **Binary version of scientific notation**
- ◆ $c \approx +2.997924 \times 10^8 \text{ m/sec}$
- ◆ $2.997924 =$ mantissa,
7 significant figures
- ◆ $+8 =$ exponent
- ◆ $.$ = *decimal* point (base 10)
- ◆ **Storage: (sign) (exponent) (mantissa)**

Model: Machine Precision

- ◆ Floating point $\Rightarrow$ limited precision
 - singles (32b): 6-7 decimal places
 - doubles 15-16 places
 - affects calculations: $7 + 10^{-7} = ?$

$7 = 0 \ 1000 \ 0010 \ 1110 \ 0000 \ 0000 \ 0000 \ 0000 \ 000,$

$10^{-7} = 0 \ 0110 \ 0000 \ 1101 \ 0110 \ 1011 \ 1111 \ 1001 \ 010$

- ◆ Different exponents $\Rightarrow$ can't add mantissas
- ◆ Shift bits (insert 0's) till same exponent

$10^{-7} = 0 \ 1000 \ 0010 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 000$

$\Rightarrow \quad 7 + 10^{-7} = 7$

- Last digits lost (truncation error)
- Ignore 10^{-7} in single precision

“Machine Precision” ϵ_m

- Measure of precision in computation
- Definition: *max* # add to 1 gives 1 as answer

$$1_c + \epsilon_m = 1_c \quad (1)$$

- $\Rightarrow$ x stored on computer:

$$\epsilon_m \approx 10^{-7} \text{ (single)}, \quad \epsilon_m \approx 10^{-16} \text{ (double)} \quad (2)$$

$$x_c \approx x (1 \pm \epsilon), \quad |\epsilon| \leq \epsilon_m \quad (3)$$

- Scientists: Just say *No! to singles*

0.000341237 =

$= 1.397706752 \times 2^{-12}$

$= 0\ 01110011\ 01100101110100000001110$

Sinal
(1 bit)

Expoente
(8 bits)

Mantissa
(23 bits)

$115 = 127 - 12$

$1.01100\dots$

- Os padrões com a mantissa $\neq 0$ e expoente = 255 não representam números, mas códigos de erro (NAN - not a number) que ocorrem, por exemplo, quando se faz uma divisão por zero.
- Numa codificação de 32 bits, não se pode armazenar mais informação do que a permitida pelos 32 bits: o número de “números reais” com representação distinta neste esquema de codificação é 2^{32} , tal como para os números inteiros.

- A interpretação dos bits é diferente, mas o número de possibilidades distintas de codificação é o mesmo.
- No caso dos números inteiros, a distribuição é uniforme, mas já o mesmo não se passa com a representação em vírgula flutuante: os “números reais” representados neste formato **NÃO** estão uniformemente distribuídos!

- De facto, a distribuição é uniforme numa escala *logarítmica*: existem tantos números (2^{24} para uma mantissa de 23 bits, ou seja cerca de 8 milhões) entre 2^{-3} e 2^{-2} como entre 2^{52} e 2^{53} !!!
- A representação dos números reais em vírgula flutuante não é (em geral) exacta e a aritmética efectuada com esta representação também não!

- Primeiro convertem-se ao mesmo expoente...
- Faz-se isto “empurrando” para a direita o número com menor expoente
- Exemplo:
$$1024 + 0.35 = 2^{10} + 1.3999999976... \times 2^{-2} =$$
$$= 2^{10} + 0.000341796... \times 2^{10}$$
- Todos os bits da mantissa que ficarem para além dos primeiros 23 são “deitados fora”...

- Em precisão simples:

$$0.35 = 1.39999997615814208984375 \times 2^{-2}$$

$$1.0110011001100110011001_2 \quad 2^{-2}$$

$$0.10110011001100110011001_2 \quad 2^{-1}$$

$$0.010110011001100110011001_2 \quad 2^0$$

$$0.0010110011001100110011001_2 \quad 2^1$$

...

$$0.000000000000101100110011001_2 \quad 2^{10}$$

$$1024 + 0.35 = (1_2 + 0.000000000000101100110011_2) \times 2^{10} \\ = 1024.3499755859375$$

$$1024 = 1024$$

$$0.35 = 0.3499999940395355224609375$$

Ao “erro de arredondamento”
adiciona-se o “erro da operação”

Casos difíceis

$$e^{-x} = \sum_{n=0}^{\infty} (-1)^n \frac{x^n}{n!}$$

$$= \sum_{n=0}^{\infty} u_n$$

Vamos ter
problemas com a divisão por
números grandes...
 $34! = 2.95233 \times 10^{38}$

$$u_n = (-1)^n \frac{x^n}{n!}$$

$$= \left[-\frac{x}{n} \right] \left[(-1)^{n-1} \frac{x^{n-1}}{(n-1)!} \right]$$

$$= -\frac{x}{n} u_{n-1}$$

- Trabalhar sempre com números da ordem de 1 (unidades “adaptadas”)
- Não somar números de ordens de grandeza muito diferentes
- Não subtrair números próximos e “grandes”
- Não dividir por números pequenos
- ...

Definição de arredondamento:

$\bar{x}$ é o valor aproximado de x , arredondado para n algarismos decimais significativos, se

$$\frac{1}{2} \cdot 10^{s-n} < |x - \bar{x}| \leq \frac{1}{2} \cdot 10^{s+1-n}$$

s é um inteiro escolhido tal que $10^s \leq x < 10^{s+1}$ (“ordem de grandeza”).

Exemplo: Arredondar π para 4 algarismos decimais significativos.

$$\pi = 3.141592653589793238462643383279502884197169399375 \dots$$

$$\pi \simeq \bar{\pi} = 3.142 \Rightarrow \Delta x = |\bar{\pi} - \pi| = 0.0004073 \dots$$

$$1 = 10^0 < 3.142 < 10 = 10^1 \Rightarrow s = 0$$

$$\Rightarrow \frac{1}{2} \cdot 10^{s-n} = 0.00005 < 0.0004073 \dots \leq 0.0005 = \frac{1}{2} \cdot 10^{s+1-n}$$

“Portugal tem 10.000.000 habitantes.”

Wow! Exatamente 10.000.000 ? Ou talvez 10.000.001 ? Ou 10.111.234 ?

Talvez seria mais adequado dizer

“Portugal tem 10 milhões de habitantes (mais do que 9, menos do que 11)”

$$N(\text{Portugal}) = 1.0 \times 10^7$$

$$INE : N(\text{Portugal}) = 10309573 \dots$$

Falta uma estimativa do erro deste número (que é uma estimativa!)

- Se $x = 1356000000$ mas $\Delta x \simeq 500000$
 $\Rightarrow$ **deve** ser apresentado $x = 1.356 \cdot 10^8$
 $x = 1356000000$ implica $\Delta x \simeq 0.5$: **errado**.

Notação de resultados com erros exactos

Se queremos apresentar um resultado com um erro mais exacto, podemos juntar o erro nos últimos algarismos em parênteses depois do número:

Exemplos:

$$x = 145.364(13) \quad \Leftrightarrow \quad x = 145.364 \pm 0.013$$

$$x = 1.356(4) \cdot 10^8 \quad \Leftrightarrow \quad x = 1.356 \cdot 10^8 \pm 0.004 \cdot 10^8$$

$$x = 145.364(5) \quad \Leftrightarrow \quad x = 145.364 \pm 0.005$$

Nota: Se o erro no último algarismo é menos de dois, costuma-se pôr mais um algarismo no resultado para especificar o erro com mais precisão. Se o erro num algarismo já é ≥ 2 , não se costuma pôr mais algarismos.

Exemplos:

$$x = 145.3642 \dots \pm 0.0134 \dots \quad \Leftrightarrow \quad x = 145.364(13)$$

$$\text{e não } x = 145.36(1)$$

$$x = 145.3642 \dots \pm 0.0343 \dots \quad \Leftrightarrow \quad x = 145.36(3)$$

$$\text{e não } x = 145.364(34)$$

$$\text{nem } x = 145.3642(343)$$

Regras de arredondamento

Em geral, utilizam-se as seguintes regras para arredondamento dos números:

- Se o primeiro dígito a desprezar for ≥ 5 , soma-se uma unidade ao último dígito a reter. Caso contrário, o último dígito a reter não será alterado.

Exemplos: de arredondamento para 5 algarismos significativos:

$$5.6170431 \simeq 5.6170$$

$$7.2346501 \simeq 7.2347$$

$$7.2346499 \simeq 7.2346$$

- O resultado do cálculos de vários números não pode ter mais algarismos significativos do que o número que tem menor quantidade deles.

Exemplos:

- $73.24 \times 4.52 = 331$ (e não 331.0448!)
 - $1.648 / 0.023 = 72$
 - $\sqrt{38.7} = 6.22$
 - $3.16 + 2.7 = 5.9$
- Para cálculos em geral aplica-se o **Lei de Gauss sobre a propagação do erro.**

Lei de Gauss sobre a propagação do erro

Sejam $x_1, x_2, \dots, x_m$ variáveis e $\Delta x_1, \Delta x_2 \dots \Delta x_m$ os seus erros.
Seja $f(x_1, x_2, \dots, x_m)$ uma função qualquer destas variáveis.

Então o erro de f pode ser estimado por

$$\Delta f \simeq \sqrt{\sum_{j=1}^m \left(\frac{\partial f}{\partial x_j} \Delta x_j \right)^2}$$